



Tutorial 5 | OS Monsoon'25

Section A

| Author → [Rachit Bhandari](#)



IPC (Inter Process Communication)

What is IPC?

- Processes are isolated (each has its own memory space).
- To collaborate, they need a way to exchange data → **IPC**.
- This is a mechanism provided by the OS which we can use to communicate between the processes
- It will allow us to exchange information and synchronise the actions between the processes

Why IPC?

1. **Data Sharing** → Processes may need to exchange data. For example, in a client-server model, the client process sends a request, and the server process responds with data.
2. **Resource Sharing** → Multiple processes might need access to shared resources like files, memory, or devices. IPC ensures orderly access and prevents race conditions.
3. **Synchronization** → Processes may need to coordinate execution, e.g., a parent process may want to wait for a child process to complete.
4. **Event Notification** → One process might need to notify another when an event occurs (e.g., a data update or completion of a task).
5. **Process Control** → A process might want to start or stop other processes or send control messages to them.

Common IPC Mechanisms

- **Signals** → low level, quite specific
 - **Pipes** → unidirectional byte streams.
 - **Shared Memory** → fastest, but needs synchronization.
 - **etc...**
-

Pipes

What is a Pipe?

- A kernel buffer with **two ends** →
 - **Write end (fd[1])**
 - **Read end (fd[0])**
- Data written at one end can be read at the other.

Key Properties

- **Unidirectional** (one-way).
- **Anonymous** (used by related processes, e.g. → parent/child).
- **Blocking** by default (read waits until data arrives).

Creating a Pipe

```
int fd[2];
if (pipe(fd) == -1) {
    perror("pipe failed :-(");
    exit(1);
}
```



Simple Example → Parent Writes, Child Reads

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    int fd[2];
    pipe(fd);

    if (fork() == 0) {
        // Child reads
        close(fd[1]);
        char buffer[100];
        read(fd[0], buffer, sizeof(buffer));
        printf("Child got: %s\n", buffer);
        exit(0);
    } else {
```

```

    // Parent writes
    close(fd[0]);
    write(fd[1], "Hello from parent!", 18);
    wait(NULL);
}
return 0;
}

```

Pipes with `exec`

Why combine `pipe()` with `exec` ?

- To build UNIX-style pipelines (`ls | grep c`).
- One process's **stdout** becomes another's **stdin**.

Example → `cat clickit | grep why`

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {

    int fd[2];
    pipe(fd);

    printf("in parent process\n\n");

    pid_t child1 = fork();

    if (child1 < 0) {
        perror("fork child1 failed");
        exit(1);
    } else if (child1 == 0) {
        printf("in first child\n\n");

        pid_t child2 = fork();
    }
}

```

```

if (child2 < 0) {
    perror("fork child2 failed");
    exit(1);
} else if (child2 == 0) {
    printf("in second child\n\n");
    printf("-----START OF EXEC BLOCK-----\n\n");
    dup2(fd[1], STDOUT_FILENO);
    close(fd[0]);
    close(fd[1]);

    char *args[] = {"cat", "clickit", NULL};
    execvp(args[0], args);
}

wait(NULL);

dup2(fd[0], STDIN_FILENO);
close(fd[1]);
close(fd[0]);

// char *args[] = {"grep", "why", NULL};
char *args[] = {"grep", "-c", "why", NULL};
execvp(args[0], args);
}

close(fd[0]);
close(fd[1]);

wait(NULL);

printf("\n-----END OF EXEC BLOCK-----\n\n");

printf("back in parent process\n\n");
return 0;
}

```